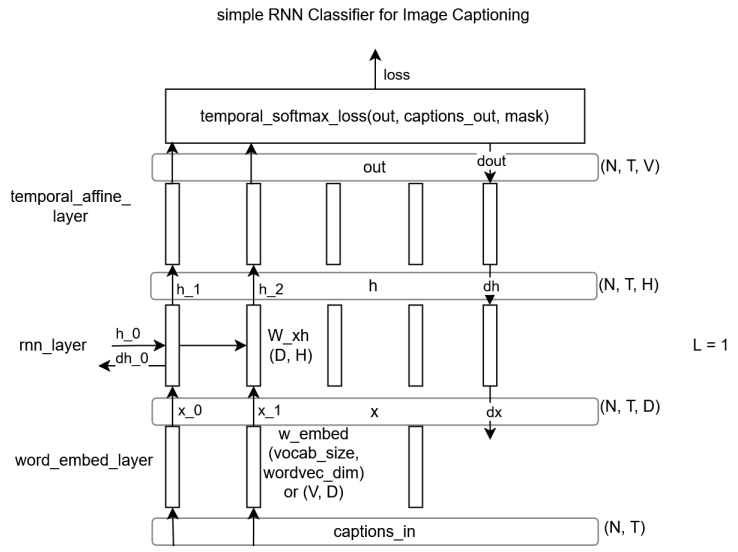
Image_Captioning

1. CNN-LSTM model for Image Captioning
    1. CNN as an encoder is used to learn features in images.
    2. LSTM as a decoder generates a text sequence describing image features.
    3. Pretrained CNN like VGG-16 or ResNet after removing the final FC classification layer acts as a feature extractor that compresses the image information into a smaller feature vector x.
    4. Input weights, $W : shape : (V, D)$, where V is the vocabulary size, captions, $\text{captions} : shape : (N, T)$ containing integers $0, \ldots, V-1$ for V vocabularies. W is indexed by captions to obtain word embeddings, x as input to LSTM which is of shape (N, T, D).
    5. example: $\text{captions} = \begin{bmatrix} 2 & 0 & 3 & 5 & 1 & 7 \\ 6 & 0 & 1 & \ldots & \ldots & \ldots \\ 2 & 2 & 3 & \ldots & \ldots & \ldots \end{bmatrix}$, $W = \begin{bmatrix} 0.1 & 0.8 & 0.025 & 0.11 \\ 0.55 & 0.2 & 0.35 & \ldots \\ 0.35 & 0.07 & 0.5 & \ldots \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$
    6. Image captioning example taken from CS231 in Neural_Networks.

```
def word_embedding_forward(x, W):
    """
    Forward pass for word embeddings. We operate on minibatches of size N where
    each sequence has length T. We assume a vocabulary of V words, assigning each
    to a vector of dimension D.

    Inputs:
    - x: Integer array of shape (N, T) giving indices of words. Each element idx
      of x muxt be in the range 0 <= idx < V.
    - W: Weight matrix of shape (V, D) giving word vectors for all words.

    Returns a tuple of:
    - out: Array of shape (N, T, D) giving word vectors for all input words.
    - cache: Values needed for the backward pass
    """
    out, cache = None, None
    ##############################################################################
    # TODO: Implement the forward pass for word embeddings.                      #
    #                                                                            #
    # HINT: This can be done in one line using NumPy's array indexing.           #
    ##############################################################################
    out = np.array([W[i, :] for i in x])
    cache = x, W
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return out, cache


def word_embedding_backward(dout, cache):
    """
    Backward pass for word embeddings. We cannot back-propagate into the words
    since they are integers, so we only return gradient for the word embedding
    matrix.

    HINT: Look up the function np.add.at

    Inputs:
    - dout: Upstream gradients of shape (N, T, D)
    - cache: Values from the forward pass

    Returns:
    - dW: Gradient of word embedding matrix, of shape (V, D).
    """
    dW = None
    ##############################################################################
    # TODO: Implement the backward pass for word embeddings.                     #
    #                                                                            #
    # Note that Words can appear more than once in a sequence.                   #
    # HINT: Look up the function np.add.at                                       #
    ##############################################################################
    x, W = cache
    dW = np.zeros(W.shape)
    np.add.at(dW, x, dout)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return dW
```



2. from show, attend and tell [4]:
    1. Extract a 14x14 Feature Map from CNN.
    2. Image: $H \times W \times 3$ -CNN-> Features, v: $L \times D$, $L = W \times H$, this Feature vector becomes an input to become an RNN's first hidden state h_0. The RNN outputs a distribution over L possible locations, a_1. These are the locations of these grids in the feature vector $L \times D$. a_1 is multiplied with the feature map to get a weighted features $z_1 = \sum_{i=1}^{L} a_i v_i$ (v_i are the features inside each grid locations that come as the output of the feature map) which will become input together with y_1 (first word) or the start token (SOS) in the next RNN time step.
    3. h_1 which is the the next hidden step of the next time step in the RNN, gives us a_2 and d_1, a distribution over vocabulary that tells us what is the predicted word. a_2 is again multiplied with the feature map to give us a new weighted representation of the image feature map z_2. And d_1 could be the input y_2, or when we're training we're training we could use the second word of the caption to be input y_2 to the next step of the RNN, so z_2 and y_2 are the inputs to hidden state to the next step in the RNN. This generates a_3 and d_2. d_2 is the distribution over the vocabulary which tells us what is the word, a_3 tells us which part of the image to focus on. This is repeated again and again, and the caption is generated.
    4. In soft attention the a_1 the weight vector, is a softmax vector, and we use it as weights to multiply with each of grid locations in v. This gives us a soft attention or the feature map of the entire image.
    5. In hard attention we only choose a winning entry and only use that grid location as an input to the next time step. Hard since we're not using a soft distribution of the weights across all parts of the image.
3. CLIP
    1. Motivation:
        1. Rather than training models with fixed set of predetermined object categories (in image classification) that limits their flexibility. Can we train a vision model to work "zero-shot"?
        2. Prior works in NLP have shown that learning from descriptions rather than fixed labels can be very data efficient. For example VirTex demonstrated data efficiency of captioning. ConVIRT showed data efficiency of contrastive training.
        3. Scale: NLP systems have benefited tremendously from scale. GPT-3 showed zero-shot transfer scale benefits.
    2. CLIP learns from text-image pairs that are already publicly available on the internet using self-supervised learning, contrastive methods, self-training approaches, and generative modeling.
    3. First key idea: use a text encoder as a classifier [5]
    4. This is an old idea -- words and pictures work goes back to ~2000, but at a smaller scale.
    5. How to scale?
        1. Learn from natural language supervision (not tags or class labels)
        2. Scrape 400 million image/text pairs
        3. "bag of words" language representation
        4. Contrastive objective, instead of predicting exact language
        5. Use transformer architecture
    6. Use small transformer language model (76M parameters for base)
    7. Matching task with large batch (size=32768)
        1. Each image and text from batch is encoded
        2. Similarity score obtained for 32k x 32k image-text pairings

3. Loss is cross-entropy on matching each image to its text, and each text to its image

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of alighed images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of iamge to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I)  #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t) / 2

"""
Numpy-like pseudocode for the core of an implementation of CLIP.
"""
```

8. Related: Pointwise Mutual Information (PMI) solver [6].
    1. The PMI solver applies associational knowledge. Given a question q and an answer option a_i, It uses pointwise mutual information (Church and Hanks 1989) to measure the strength of the associations between parts of q and parts of a_i. Given a large corpus C, PMI for two n-grams x and y is defined as: $PMI(x, y) = \log \frac{p(x,y)}{p(x)p(y)}$

References

1. Paper: An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition - Baoguang Shi et al
2. Convolutional Image Captioning, Jyoti Aneja, Aditya Deshpande, Alexander G. Schwig.
3. Karpathy et al, Deep visual-semantic alignments for generating image description, CVPR 2015.
4. Xu et al, Show attend and tell: Neural Image Caption generation with Visual Attention, ICML 2015.
5. CS441, Applied Machine Learning, Derek Hoiem, 2023.
6. Clark, P., Etzioni, O., Khot, T., Sabharwal, A., Tafjord, O., Turney, P., & Khashabi, D. (2016). Combining Retrieval, Statistics, and Inference to Answer Elementary Science Questions. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*(1). https://doi.org/10.1609/aaai.v30i1.10325